SOFTWARE ENGINEERING MINI PROJECT · TEAM E15

Software Requirements Specification

Railway Reservation System

Prepared by Mohit Singh P

PES1UG24CS273

Version 1.1 · 29 September 2026

ISO/IEC/IEEE 29148-aligned structure

SUBMISSION COPY

Document Control

Document	Software Requirements Specification (SRS)
System	Railway Reservation System (RRS)
Owner	Mohit Singh P — PES1UG24CS273
Team	Team E15
Version / status	1.1 / Submission copy
Date	29 September 2026

Revision History

Version	Date	Change	Author
1.0	29 Sep 2026	Initial Part 1 requirements baseline.	Mohit Singh P
1.1	29 Sep 2026	Separated submission document; strengthened traceability, verification wording and UML presentation.	Team E15 review

Baseline rule. “Shall” statements are mandatory and verifiable. A change to any FR, NFR or SEC identifier requires review of the architecture mapping and test traceability matrix.

Table of Contents

1. Introduction 4
    1.1 Purpose 4
    1.2 Scope 4
    1.3 Intended audience and conventions 4
    1.4 Definitions and references 4
2. Overall Description 4
3. External Interface Requirements 5
4. Specific Requirements 6
    4.1 Functional requirements 6
    4.2 Non-functional requirements 6
    4.3 Security objectives and requirements 7
5. UML Use Cases 8
6. Acceptance Basis 9

1. Introduction

1.1 Purpose

This specification defines the observable behavior, interfaces, constraints, quality attributes and security requirements of the Railway Reservation System prototype. It is the requirements baseline for architecture, design and testing.

1.2 Scope

Passengers register, sign in, search trains, inspect seat availability, create a reservation after simulated payment, view their bookings and cancel an eligible booking. Administrators maintain train trips and seat capacity and review audit events. The prototype excludes live railway-network integration, real money movement, dynamic pricing and wait-list allocation.

1.3 Intended Audience and Conventions

This document is intended for the project team, instructors, reviewers and testers. Requirement identifiers use **FR** for functional requirements, **NFR** for non-functional requirements and **SEC** for security requirements. Times are measured at the service boundary under the Test Plan's stated conditions.

1.4 Definitions and References

Term	Definition
PNR	A unique booking reference.
Seat hold	A temporary exclusive claim on one seat during a booking transaction.
Confirmed booking	A booking with one allocated seat and successful simulated payment.
Administrator	An authenticated user authorized to manage schedules, inventory and audit review.
Idempotency key	A caller-supplied unique key that makes a retried booking request return the original result rather than create another booking.

References: SE Mini Project Deliverables Part-1; ISO/IEC/IEEE 29148:2018 requirements-engineering guidance.

2. Overall Description

2.1 Product Perspective

RRS is a browser-based client backed by a REST API and relational database. The browser never accesses the database directly. A deterministic payment simulator provides success, failure and refund responses for testing.

2.2 Product Functions

- Account registration, authentication, sign-out and lockout.
- Train search, ordered results and current seat availability.
- Idempotent booking, simulated payment and unique PNR generation.
- Passenger-owned booking history and eligible cancellation.
- Administrator trip/inventory management and audit review.

2.3 User Classes

User class	Permitted responsibilities
Passenger	Manage own account session; search; view availability; book; view and cancel own eligible bookings.
Administrator	Create, edit and disable trips and capacity; review audit events. Administrator permissions are enforced server-side.

2.4 Operating Environment

A current desktop or mobile browser connects to the application over HTTPS. The API and database run as separate server processes. Timestamps are stored in UTC and displayed with an explicit timezone label.

2.5 Assumptions and Constraints

- A trip has a train number, origin, destination, departure/arrival time, travel date, fare and finite capacity.
- One seat is booked per transaction. A cancellation is eligible only before departure.
- Test accounts, seeded trip data and configurable payment-simulator responses are available.
- No real payment credentials or real passenger identities are processed.

3. External Interface Requirements

3.1 User Interfaces

Passenger screens support registration, sign-in, search, availability, checkout, booking history and cancellation. Administrator screens support trip management and audit review. Forms identify required fields, expose programmatic labels and show actionable validation messages.

3.2 Software Interfaces

The UI exchanges JSON with the REST API over HTTPS. The payment simulator exposes authorization and refund operations. The database persists users, trips, seats, bookings, payment events and audit events.

3.3 Communication Interfaces

Authenticated traffic uses HTTPS. JSON dates use ISO 8601 with an explicit UTC offset. Sessions are represented by secure tokens whose lifecycle is defined in SEC-03.

4. Specific Requirements

4.1 Functional Requirements

> **FR-01 — Registration.** The system shall create a passenger account when a unique email, a name and a password of at least 12 characters are submitted. It shall reject an 11-character password and a duplicate email without creating an account.

> **FR-02 — Authentication.** The system shall issue an authenticated session only after valid credentials are supplied and shall reject an invalid password with a generic error that does not reveal whether an account exists.

> **FR-03 — Train search.** Given an origin, destination and travel date, the system shall return only trips matching all three values, ordered by departure time.

> **FR-04 — Availability.** For every returned trip, the system shall show the number of unreserved seats; the displayed count shall remain between zero and the trip capacity inclusive.

> **FR-05 — Booking.** When an authenticated passenger selects an available trip and the payment simulator returns success, the system shall allocate exactly one available seat and create exactly one confirmed booking with a unique PNR. If payment fails, it shall release the hold and create no confirmed booking. Retrying the same idempotency key shall not create another booking.

> **FR-06 — Booking history.** The system shall show a passenger only that passenger’s bookings, including PNR, trip, seat, fare and status.

> **FR-07 — Cancellation.** For a confirmed booking owned by the signed-in passenger and requested before departure, the system shall mark it cancelled, release its seat exactly once and record a simulated refund reference. It shall reject late, repeated and non-owner cancellation attempts without a second inventory change.

> **FR-08 — Administration.** An administrator shall create, edit or disable a train trip and its seat capacity; a non-administrator request to these operations shall receive HTTP 403.

> **FR-09 — Audit.** The system shall record successful and failed sign-ins, booking confirmations, booking cancellations and administrator trip changes with timestamp, actor identifier, action and outcome.

4.2 Non-Functional Requirements

> **NFR-01 — Search performance.** With 10,000 stored trips and 50 concurrent users, 95% of 200 valid search requests shall complete within 2 seconds in the stated test environment.

> **NFR-02 — Booking performance.** With 50 concurrent users, 95% of 100 booking requests shall complete within 3 seconds, excluding any injected simulator delay above 1 second.

> **NFR-03 — Concurrency.** When 20 concurrent requests attempt to reserve the final seat, at most one booking shall be confirmed and availability shall finish at zero, never below zero.

> **NFR-04 — Availability and recovery.** After an application-process restart, committed bookings shall remain available and the service shall answer a health request within 60 seconds.

> **NFR-05 — Accessibility.** Required passenger and administrator workflows shall support keyboard-only operation and every input and button shall expose an accessible programmatic name.

> **NFR-06 — Compatibility.** Required workflows shall work in the latest stable Chrome, Edge and Firefox at 375-pixel and 1280-pixel viewport widths without horizontal scrolling of primary forms.

4.3 Security Objectives and Requirements

Security Objectives

- **SO-01:** Prevent unauthorized access to passenger records and administrator functions.
- **SO-02:** Protect credentials and booking data in transit and at rest.
- **SO-03:** Preserve a trustworthy audit history of security-sensitive actions.

Security Requirements

> **SEC-01 — Access control.** Every booking read, cancellation and administrator API request shall validate the authenticated user and role on the server. A request for another passenger’s booking shall return HTTP 403 or 404 without booking details.

> **SEC-02 — Credential protection.** Passwords shall be stored only as salted adaptive hashes. Plaintext passwords shall not appear in the database, logs or API responses.

> **SEC-03 — Transport and session.** Authenticated API traffic shall use HTTPS. Session tokens shall expire after 30 minutes of inactivity and be invalid after sign-out.

> **SEC-04 — Abuse resistance and audit.** After five failed sign-in attempts for one account within 15 minutes, further attempts shall be rejected for 15 minutes and logged without the submitted password.

5. UML Use Cases

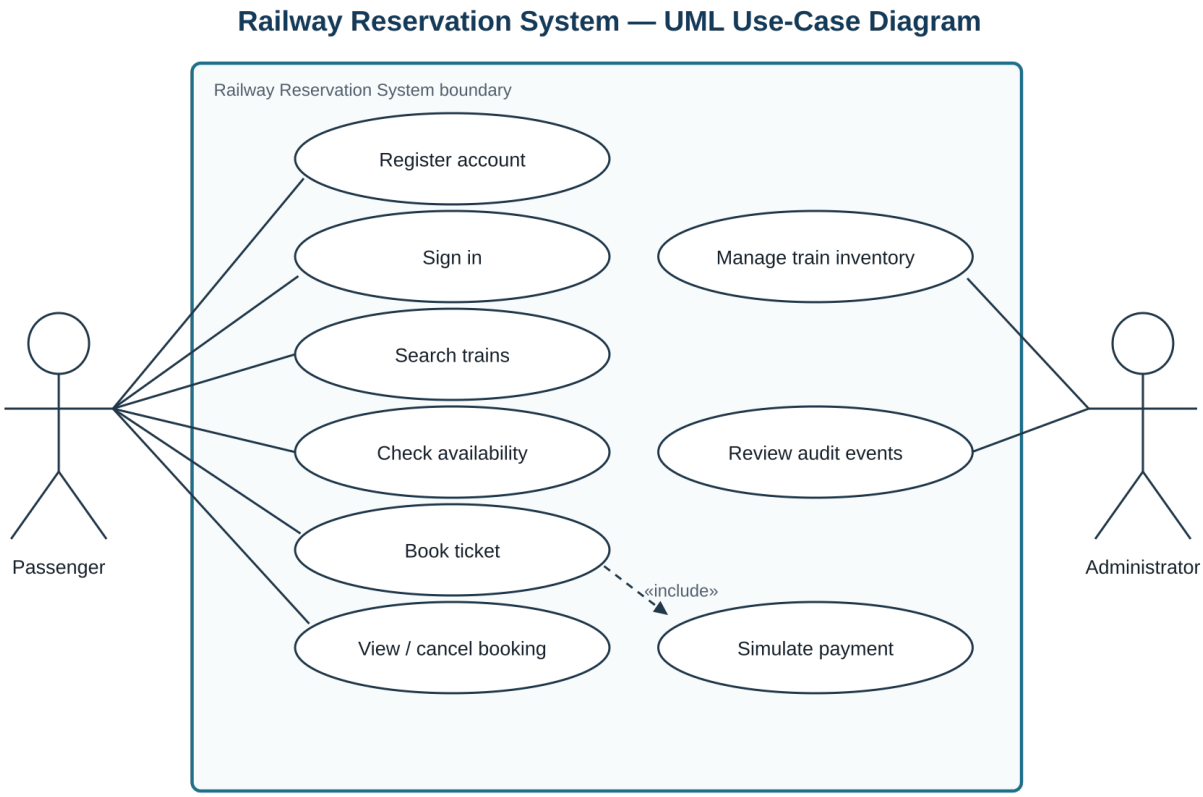


Figure 1. Passenger and administrator use cases with separate UML actor associations.

5.1 UC-01 — Book Ticket

Primary actor	Passenger
Preconditions	The passenger is authenticated; the selected trip is enabled.
Main flow	Search trip → check availability → submit booking with idempotency key → reserve one seat → simulate payment → create confirmed booking and PNR → record audit event.
Alternatives	Sold-out inventory returns conflict. Payment failure releases the hold. A duplicate idempotency key returns the original result.
Postcondition	Exactly one confirmed booking exists on success; none exists after payment failure.

5.2 UC-02 — Cancel Booking

Primary actor	Passenger
Preconditions	The booking belongs to the passenger, is confirmed and its trip has not departed.
Main flow	Select booking → validate ownership/status/time → cancel once → release seat once → record refund reference and audit event.
Alternatives	Non-owner receives 403/404. Late or repeated cancellation is rejected without an additional seat change.
Postcondition	The booking is cancelled and the inventory change is idempotent.

6. Acceptance Basis

The SRS baseline is acceptable for Part 1 when every FR, NFR and SEC identifier maps to at least one planned test, the required UML diagrams are present across the submission, and the booking/cancellation invariants are explicitly verified. The Test Plan’s requirement traceability matrix is the authoritative coverage record.

SOFTWARE ENGINEERING MINI PROJECT · TEAM E15

Test Plan

Railway Reservation System

Prepared by Atharv More

PES1UG24CS276

Test-case contribution: Muhammad Affan Dumba · PES1UG24CS280

Version 1.1 · 29 September 2026

ISO/IEC/IEEE 29119-3-aligned structure

SUBMISSION COPY

Document Control

Document	Test Plan
System	Railway Reservation System (RRS)
Plan owner	Atharv More — PES1UG24CS276
Test-case owner	Muhammad Affan Dumba — PES1UG24CS280
Team	Team E15
Version / status	1.1 / Submission copy
Date	29 September 2026

Revision History

Version	Date	Change	Author
1.0	29 Sep 2026	Initial Part 1 plan and ten case specifications.	Atharv More / Muhammad Affan Dumba
1.1	29 Sep 2026	Added explicit password-boundary/hash, late-cancellation and booking/cancellation-audit checks; separated submission copy.	Team E15 review

Execution status. The cases in §5.2 are specifications, not claims of execution. Actual result, evidence, tester, date, build and status are recorded when an implementation is available.

Table of Contents

1. Introduction 4
2. Test Items and Scope 4
3. Test Strategy and Approach 4
4. Environment, Responsibilities and Schedule 5
5. Test Design and Validation 6
    5.1 Security validation 6
    5.2 Test cases 7
6. Completion and Deliverables 10
7. Requirement Traceability Matrix 11

1. Introduction

1.1 Purpose

This plan defines the scope, strategy, resources, environment, entry/exit criteria, security validation and traceability for system testing of RRS. The SRS v1.1 is the requirements baseline.

1.2 Objectives

- Verify all functional, non-functional and security requirements.
- Verify booking/cancellation invariants under normal, failure and concurrent conditions.
- Produce repeatable evidence and a requirement-level completion review.

1.3 References

Team E15 RRS SRS v1.1; Team E15 Architecture and Design Specification v1.1; ISO/IEC/IEEE 29119-3:2021 test documentation guidance.

2. Test Items and Scope

In scope	Registration; authentication; search; availability; booking; history; cancellation; administration; audit; performance; concurrency; restart recovery; accessibility; compatibility; transport, session, credential and authorization security.
Test items	Browser UI, REST API, domain services, relational database behavior and payment-simulator integration.
Out of scope	Real card payments, live railway feeds, production disaster recovery, dynamic pricing and wait-list allocation.
Assumptions	Seeded synthetic data, test-only database inspection, configurable payment outcomes and current supported browsers are available.

3. Test Strategy and Approach

3.1 Test Levels

- **Unit:** validation, password policy, state transitions and availability calculations.
- **API integration:** authorization, database constraints, persistence, idempotency and simulator outcomes.
- **System:** passenger/administrator browser workflows, accessibility and compatibility.
- **Non-functional:** load, concurrency and restart recovery.

3.2 Techniques

Positive/negative scenarios, equivalence partitions, boundary values, role/ownership checks, duplicate submission, state-transition testing and concurrent final-seat requests are used. Automated checks are preferred for API, load and security work; keyboard and visual compatibility checks may be manual with recorded evidence.

3.3 Test Data

Use synthetic passenger accounts P1 and P2, administrator A1, trips with zero/one/multiple seats, a departed trip, success/failure simulator modes and a 10,000-trip performance seed. Reset independent cases to a known database snapshot.

3.4 Defect Handling

Record case ID, build, environment, reproducible steps, expected/actual results, severity and evidence. Retest a fix and execute affected regression cases. Critical or high defects block exit unless a documented academic-review decision states otherwise.

4. Environment, Responsibilities and Schedule

4.1 Environment

- HTTPS test deployment and isolated relational test database.
- Current stable Chrome, Edge and Firefox; 375-pixel and 1280-pixel viewports.
- Load/concurrency tool supporting at least 50 virtual users and synchronized requests.
- Authorized, read-only database/log inspection for credential and audit validation.

4.2 Responsibilities

Member	Responsibility
Atharv More	Test Plan ownership, environment coordination, execution tracking and exit review.
Muhammad Affan Dumba	§5.2 test-case specifications, coverage maintenance and evidence checklist.
Mohit Singh P	Requirement mapping review.
Mohammed Rizwan Shaikh	Architecture-related setup, interface and failure-behavior review.

4.3 Sequence

Baseline documents → deploy/seed → smoke test → functional/security cases → performance/ concurrency → accessibility/compatibility → defect retest/regression → traceability and exit review.

4.4 Entry and Exit Criteria

Entry	SRS and design baselined; test build deployed; simulator and seed data available; test accounts and evidence repository prepared.
Exit	All ten cases have recorded outcomes; no open critical/high defects; every FR/NFR/SEC statement is demonstrably covered or has a documented follow-up test; evidence is reviewed.

4.5 Risks and Mitigations

Risk	Mitigation
Environment timing distorts performance	Record hardware/build/tool versions; warm up; repeat runs using the same seed.
30-minute session test is slow	Run the real-duration acceptance check; use configurable time only for earlier regression.
Concurrent requests are not synchronized	Use a barrier so all 20 final-seat requests start together.
Sensitive test data leaks	Use synthetic identities and redact tokens/passwords from evidence.

5. Test Design and Validation

A case passes only when every expected result is observed and retained evidence supports the result. Failure of one assertion fails the case until corrected and rerun.

5.1 Security Validation

ID	Validation	Cases
SV-01	Verify generic authentication failure, fifth-attempt threshold, 15-minute rejection window and password-free audit records.	TC-02
SV-02	Use P2 against P1’s booking and P1 against administrator endpoints; expect 403/404 with no protected data.	TC-06, TC-07
SV-03	Inspect HTTPS, replay a signed-out token and test expiry after 30 minutes of inactivity.	TC-09
SV-04	Inspect authorized test storage/logs: password is a salted adaptive hash and plaintext is absent from database, API responses and logs.	TC-01, TC-02

5.2 Test Cases

TC-01 — Registration, Password Boundary and Credential Storage

Preconditions	No P1 account; authorized test-only read access to user storage and logs.
Procedure	Submit an 11-character password; submit a valid 12-character password and unique email; repeat the same email; inspect stored credential, response and logs.
Expected	11-character input is rejected; valid input creates one account; duplicate is rejected; stored value is a salted adaptive hash and does not equal the password; plaintext appears in neither responses nor logs.
Traceability	FR-01; SEC-02
Evidence	API/UI responses, redacted storage inspection and log search.

TC-02 — Sign-In, Lockout and Authentication Audit

Preconditions	P1 exists and is unlocked.
Procedure	Sign in correctly and sign out; submit five incorrect passwords within 15 minutes; attempt a sixth; inspect audit; retry after the 15-minute rejection period.
Expected	Valid login works; invalid responses are generic; sixth attempt is rejected for 15 minutes; success, failures and rejection are audited without submitted passwords; access resumes after the period with valid credentials.
Traceability	FR-02, FR-09; SEC-02, SEC-04
Evidence	Timestamped responses and redacted audit records.

TC-03 — Search and Availability

Preconditions	Matching, nonmatching, sold-out and multiple-departure trips are seeded.
Procedure	Search by origin, destination and date; inspect filtering, ordering and every availability value.
Expected	Only exact route/date matches appear in departure order; every count is between zero and capacity.
Traceability	FR-03, FR-04
Evidence	Seed extract and ordered API/UI result.

TC-04 — Successful Booking, History and Audit

Preconditions	P1 signed in; selected trip has one free seat; simulator succeeds.
Procedure	Submit one booking; query P1 history and audit records.
Expected	Exactly one confirmed booking with unique PNR and one allocated seat; P1 history shows PNR/trip/seat/fare/status; confirmation audit contains actor, action, time and outcome.
Traceability	FR-05, FR-06, FR-09
Evidence	Booking response, database state, history and audit event.

TC-05 — Payment Failure and Idempotent Retry

Preconditions	P1 signed in; one free seat; known idempotency key.
Procedure	Run with simulator failure; verify state. Then configure success and send the same logical request twice with one new idempotency key.
Expected	Failure confirms nothing and releases the hold; duplicate success calls return one booking/PNR and allocate one seat.
Traceability	FR-05; NFR-03
Evidence	Responses, booking count and inventory before/after.

TC-06 — Ownership, Eligible/Late Cancellation and Audit

Preconditions	P1 owns one future confirmed booking and one departed confirmed booking; P2 signed in separately.
Procedure	P2 reads/cancels P1 future booking; P1 cancels it twice; P1 attempts to cancel the departed booking; inspect inventory/refund/audit.
Expected	P2 gets 403/404 without details; P1’s first eligible cancellation succeeds with one seat release/refund reference/audit event; repeat and late attempts are rejected without further inventory change.
Traceability	FR-06, FR-07, FR-09; SEC-01
Evidence	Responses, inventory deltas, payment event and audit events.

TC-07 — Administrator Inventory, Authorization and Audit

Preconditions	A1 administrator and P1 passenger exist.
Procedure	P1 attempts administrator create; A1 creates, edits and disables a trip; inspect state and audit.
Expected	P1 receives 403; A1 changes persist; each change has actor, timestamp, action and outcome.
Traceability	FR-08, FR-09; SEC-01
Evidence	Responses, final trip state and audit events.

TC-08 — Concurrent Final Seat

Preconditions	Exactly one seat; 20 distinct authenticated passengers; simulator succeeds.
Procedure	Release 20 booking requests simultaneously from a synchronization barrier; inspect bookings and availability.
Expected	At most one confirmation; others report unavailable; final count is zero and is never negative.
Traceability	FR-04, FR-05; NFR-03
Evidence	Tool timestamps, responses, booking count and availability history.

TC-09 — Transport, Session and Restart Recovery

Preconditions	HTTPS deployment; P1 session; committed booking exists.
Procedure	Inspect transport; sign out and replay token; create a session, wait 30 minutes idle and reuse it; restart application; query health and booking.
Expected	Authenticated traffic uses HTTPS; signed-out/expired tokens fail; booking persists; health responds within 60 seconds; logs omit tokens/passwords.
Traceability	NFR-04; SEC-02, SEC-03
Evidence	Transport capture, timestamped responses, restart log and persisted record.

TC-10 — Performance, Accessibility and Compatibility

Preconditions	10,000 trips; 50 virtual users; Chrome/Edge/Firefox; 375px and 1280px viewports.
Procedure	Run 200 searches and 100 bookings at 50 users; complete required workflows by keyboard at both widths in each browser; inspect accessible names.
Expected	≥95% searches ≤2s; ≥95% bookings ≤3s under the simulator condition; keyboard completion succeeds; every required control has an accessible name; no primary-form horizontal scrolling.
Traceability	NFR-01, NFR-02, NFR-05, NFR-06
Evidence	Load summary, browser/version matrix, screenshots and accessibility report.

6. Test Completion and Deliverables

Evidence includes completed case records, API responses/screenshots, performance summaries, accessibility notes, audit/storage inspection, defect log and final traceability review. A failed case is not waived without a recorded decision and reason.

Execution metadata required per case	Build, environment, tester, date/time, actual result, evidence link, pass/fail/block status and defect ID where applicable.
Completion review	Test owner and requirement reviewer confirm evidence and unresolved risks against exit criteria.

7. Requirement Traceability Matrix

Requirement	Planned verification
FR-01	TC-01
FR-02	TC-02
FR-03	TC-03
FR-04	TC-03, TC-08
FR-05	TC-04, TC-05, TC-08
FR-06	TC-04, TC-06
FR-07	TC-06
FR-08	TC-07
FR-09	TC-02, TC-04, TC-06, TC-07
NFR-01	TC-10
NFR-02	TC-10
NFR-03	TC-05, TC-08
NFR-04	TC-09
NFR-05	TC-10
NFR-06	TC-10
SEC-01	TC-06, TC-07
SEC-02	TC-01, TC-02, TC-09
SEC-03	TC-09
SEC-04	TC-02

SOFTWARE ENGINEERING MINI PROJECT · TEAM E15

Software Architecture and Design Specification

Railway Reservation System

Prepared by Mohammed Rizwan Shaikh

PES1UG24CS272

Version 1.1 · 29 September 2026

IEEE 1016-aligned structure

SUBMISSION COPY

Document Control

Document	Software Architecture and Design Specification
System	Railway Reservation System (RRS)
Owner	Mohammed Rizwan Shaikh — PES1UG24CS272
Team	Team E15
Version / status	1.1 / Submission copy
Date	29 September 2026

Revision History

Version	Date	Change	Author
1.0	29 Sep 2026	Initial Part 1 architecture and design.	Mohammed Rizwan Shaikh
1.1	29 Sep 2026	Separated document; corrected UML dependencies and booking-service orchestration; standardized author name.	Team E15 review

Design authority. The Booking Service owns booking and cancellation transactions. The REST API authenticates, authorizes, validates and delegates; it does not bypass the domain service to write booking state.

Table of Contents

1. Introduction and Design Goals 4
2. Architecture 4
    2.1 Architectural pattern 4
    2.2 Component view and descriptions 5
    2.3 Data and deployment view 6
    2.4 Security architecture 6
    2.5 Architecture traceability 6
3. Detailed Design 7
    3.1 Booking sequence 7
    3.2 Cancellation sequence 8
    3.3 API design 9
    3.4 Error handling and state rules 9
    3.5 Design verification 9
4. References and Change Control 10

1. Introduction and Design Goals

This document defines the architecture and detailed design for the Railway Reservation System described by the SRS. The design uses a browser client, REST API, domain services, relational database and deterministic payment simulator.

1.1 Design Goals

- Preserve one confirmed booking for one successful idempotent request.
- Prevent double allocation and negative seat availability under concurrency.
- Enforce role and booking ownership on the server, independent of the UI.
- Keep security-sensitive operations auditable without recording secrets.
- Separate presentation, API, domain and persistence responsibilities.

1.2 Scope and Constraints

The design covers the Part 1 prototype and simulated payment/refund behavior. It does not define a production payment gateway, live railway integration or wait-list allocation. All components must preserve the requirement identifiers defined in the SRS.

2. Architecture

2.1 Architectural Pattern

A layered client–server pattern separates the Web UI, REST API/authorization boundary, domain services and persistence. The browser never accesses the database directly. Domain services own business state transitions. The payment simulator is isolated behind an adapter so it can later be replaced without changing passenger workflows.

Transaction boundary. Seat locking, payment outcome handling, booking confirmation and idempotency are coordinated by the Booking Service. Database uniqueness constraints remain the final concurrency guard.

2.2 Component View and Descriptions

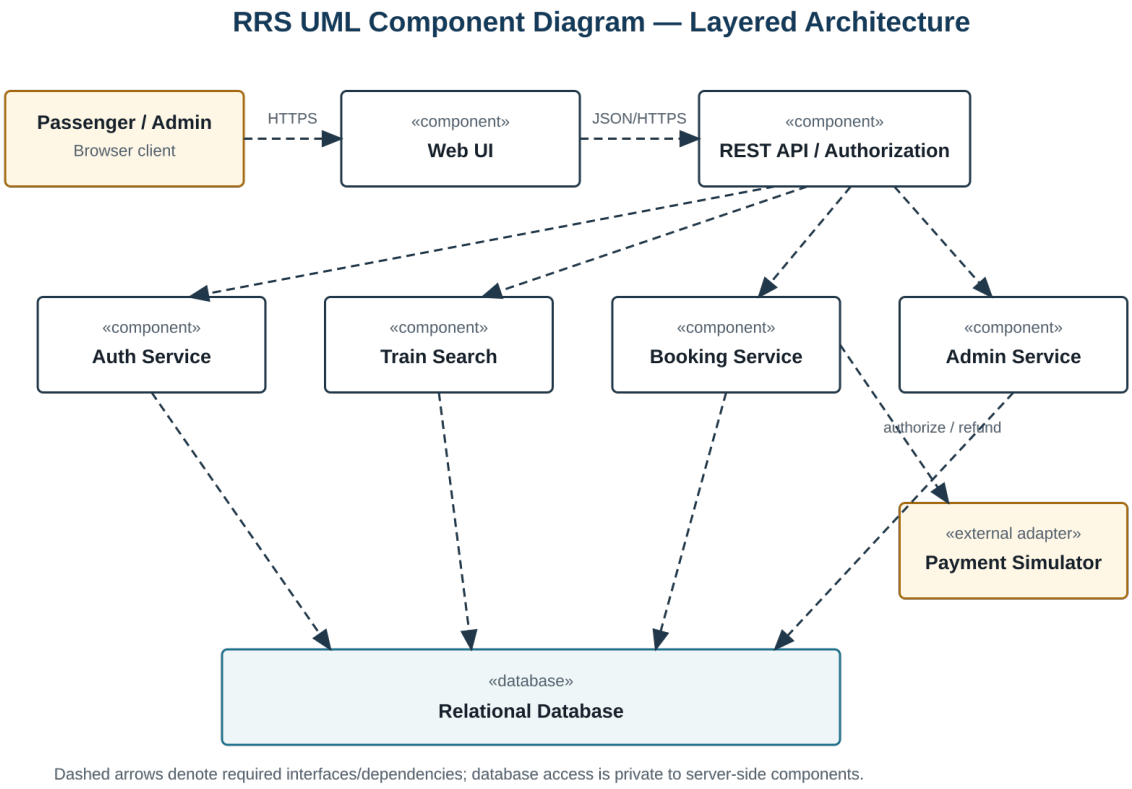


Figure 1. Layered component dependencies and protected database boundary.

Component	Responsibilities	Primary requirements
Web UI	Collect inputs, expose accessible controls, display state and validation, call only the REST API.	NFR-05–06
REST API / Authorization	Authenticate requests, validate payloads, enforce roles and ownership, route to domain services, assign request IDs.	FR-02, SEC-01, SEC-03
Auth Service	Registration, adaptive password hashing, credential verification, session expiry and lockout.	FR-01–02, SEC-02–04
Train Search	Match route/date filters, order departures and calculate valid availability.	FR-03–04, NFR-01
Booking Service	Seat holds, idempotent confirmation, PNR generation, cancellation, refund status and booking audit.	FR-05–07, FR-09, NFR-02–04
Admin Service	Create, edit and disable trips/capacity and append administrative audit events.	FR-08–09
Payment Simulator	Return deterministic authorization/refund outcomes and references for testing.	FR-05, FR-07
Relational Database	Persist state; enforce unique email/PNR/idempotency/seat allocation and transactional integrity.	FR-01, FR-04–09, NFR-03–04

2.3 Data and Deployment View

Entity	Key fields and constraints
User	`id, email, password_hash, role`; unique normalized email; no plaintext password field.
Trip	`id, train_number, origin, destination, departure_utc, arrival_utc, fare, capacity, enabled`.
Seat	`id, trip_id, seat_number`; unique `(trip_id, seat_number)`.
Booking	`id, pnr, user_id, trip_id, seat_id, status, idempotency_key`; unique PNR, unique confirmed seat, unique caller/idempotency key.
PaymentEvent	`id, booking_id, kind, status, reference`; contains no payment credentials.
AuditEvent	`id, actor_id, action, outcome, created_utc, request_id`; append-only at the application boundary.

The Web UI is served to current browsers. API/domain services and the database run in separate processes. Database network access is private to server-side services. HTTPS terminates at the trusted application edge, and authenticated traffic remains protected across any internal hop.

2.4 Security Architecture

- **Identity:** salted adaptive password hashes; generic authentication errors; account lockout after the defined threshold.
- **Authorization:** the API and domain layer both verify actor, role and resource ownership.
- **Sessions:** HTTPS-only authenticated traffic, inactivity expiry and server-side invalidation on sign-out.
- **Data protection:** parameterized database operations, private database access and no secrets in errors or audit payloads.
- **Auditability:** actor, action, timestamp, request ID and outcome are recorded for defined events; submitted passwords and tokens are excluded.
- **Concurrency:** transaction locks plus uniqueness constraints prevent two confirmed bookings for the same seat.

2.5 Architecture Traceability

Requirements	Design realization
FR-01–02; SEC-02–04	Auth Service, API security boundary and AuditEvent.
FR-03–04; NFR-01	Train Search, indexed trip data and calculated availability.
FR-05–07; NFR-02–04	Booking Service, Payment Simulator and transactional database constraints.
FR-08	API role checks and Admin Service.
FR-09	Auth, Booking and Admin services append AuditEvent records.
SEC-01	API plus domain-layer role/ownership checks.
NFR-05–06	Accessible responsive Web UI.

3. Detailed Design

3.1 Booking Sequence

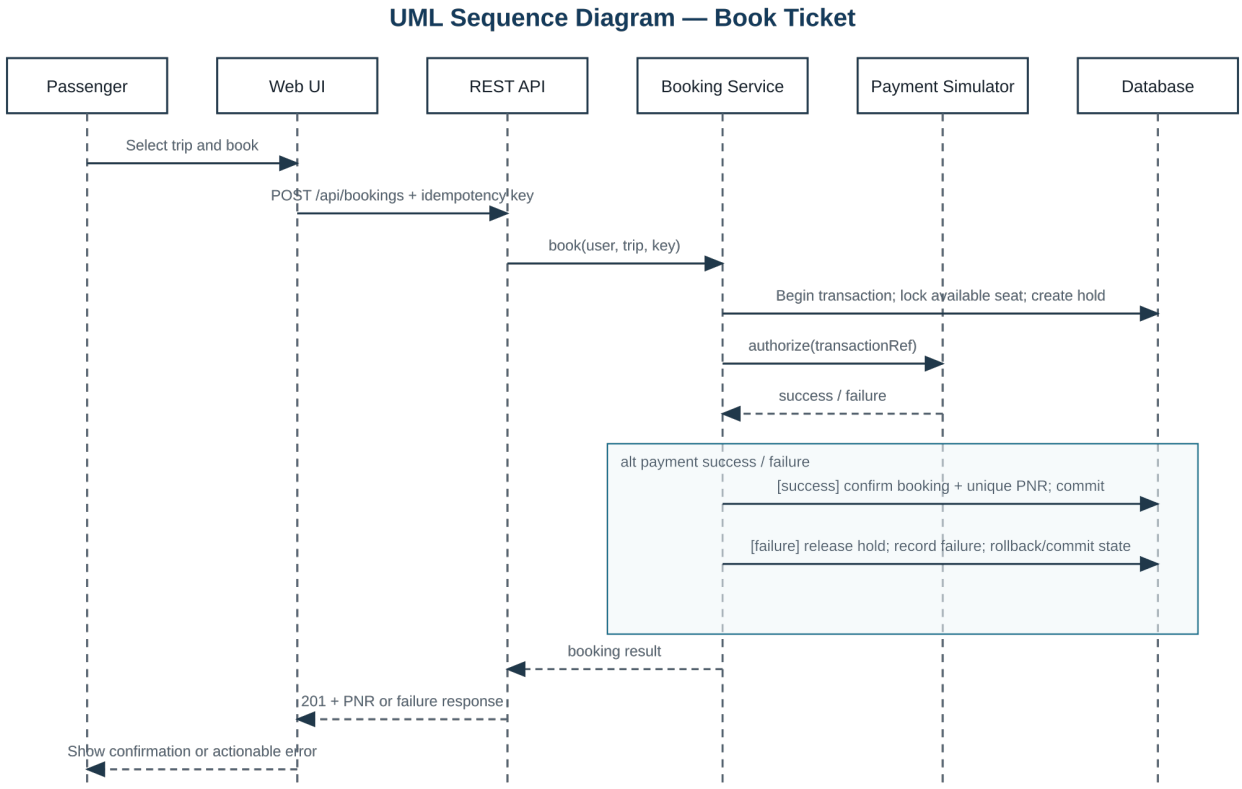


Figure 2. Booking Service consistently owns the booking transaction and payment outcome.

The service obtains or returns the existing idempotent result, locks one available seat and creates a temporary hold. It calls the simulator with a transaction reference. Success confirms the booking and PNR in the same controlled workflow; failure releases the hold and creates no confirmed booking.

3.2 Cancellation Sequence

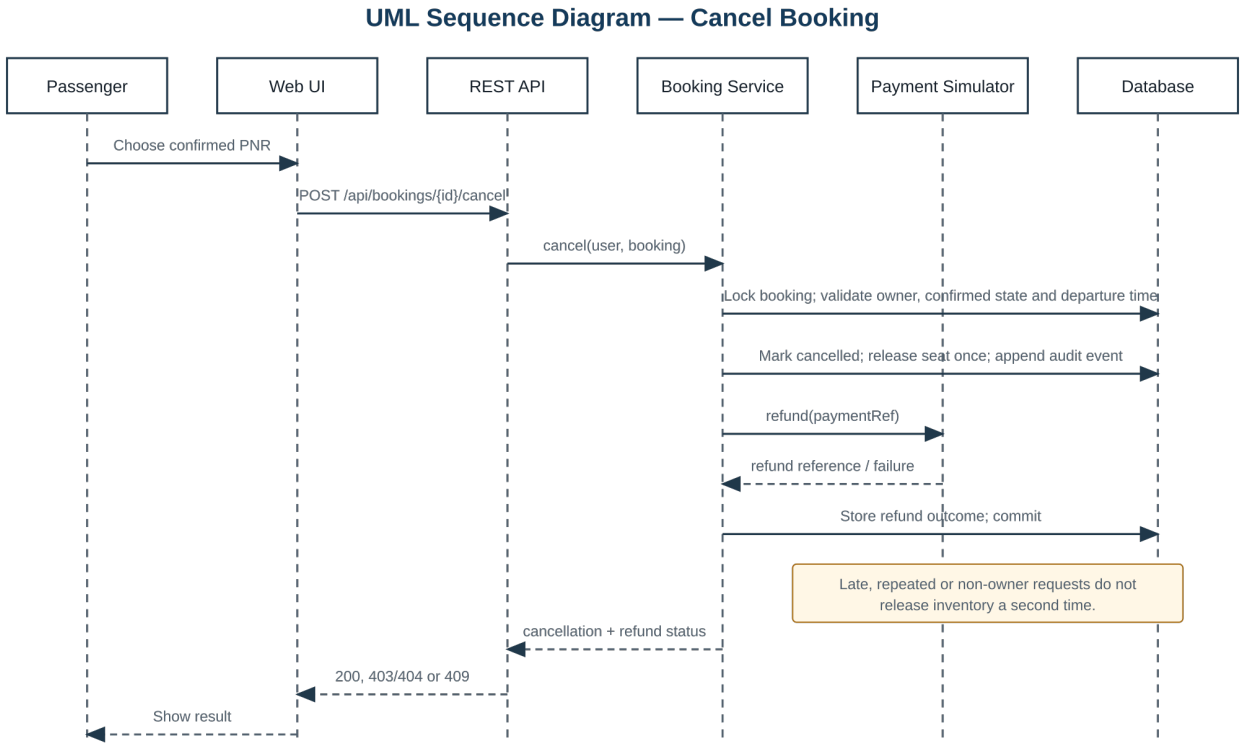


Figure 3. Cancellation checks owner, state and departure before a single inventory release.

A confirmed booking moves to cancelled once. Late, repeated and non-owner requests do not change inventory. Refund failure remains visible as a payment-event outcome without restoring the cancelled booking.

3.3 API Design

Method and resource	Success	Principal failures
`POST /api/register`	201 account created	400 validation; 409 duplicate
`POST /api/sessions`	200 session	401 invalid; 429 locked
`DELETE /api/sessions/current`	204	401 unauthenticated
`GET /api/trips?origin=&destination=&date=`	200 ordered trips	400 invalid query
`GET /api/trips/{id}/availability`	200 availability	404 missing
`POST /api/bookings` + `Idempotency-Key`	201 PNR/seat/status	402 simulated payment failure; 409 unavailable
`GET /api/bookings`	200 caller-owned records	401 unauthenticated
`POST /api/bookings/{id}/cancel`	200 cancellation/refund status	403/404 non-owner; 409 late/ineligible
`POST /api/admin/trips`	201	403 non-admin; 409 conflict
`PATCH /api/admin/trips/{id}`	200	403; 404; 409
`DELETE /api/admin/trips/{id}`	204 disable	403; 404
`GET /api/admin/audit`	200 events	403 non-admin

Request and response bodies are JSON. Dates use ISO 8601 with an explicit UTC offset. Mutating operations attach a request ID to their audit event.

3.4 Error Handling and State Rules

Errors use `{"code":"...","message":"...","requestId":"..."}`. Validation uses 400, unauthenticated 401, forbidden 403, missing 404, state conflict 409, lockout 429 and unexpected failure 500. Responses never contain passwords, tokens, SQL or stack traces.

State rule	Enforcement
Booking: pending → confirmed or failed	Transaction-controlled; failed cannot become confirmed without a new request.
Booking: confirmed → cancelled	Owner, state and departure checks; no transition back to confirmed.
Seat allocation	Lock plus database uniqueness; transaction rollback prevents partial allocation.
Duplicate request	Caller/idempotency-key uniqueness returns the stored outcome.

3.5 Design Verification

TC-04, TC-05 and TC-08 verify booking transactions, idempotency and concurrency. TC-06 verifies ownership, eligibility, cancellation idempotency and booking audit. TC-07 verifies administrator separation and audit. TC-09 verifies session and restart behavior. Database constraints and endpoint contracts are reviewed against their mapped SRS identifiers before implementation.

4. References and Change Control

References: Team E15 RRS SRS v1.1; IEEE 1016:2009 software design description guidance; ISO/IEC/IEEE 29119-3:2021 test documentation guidance.

Any change to an FR, NFR or SEC identifier requires review of component mapping, API contracts, state invariants, security controls and test coverage.